

Detección de Jugadores, Arqueros, Árbitros y Pelota en Video de Transmisiones de Fútbol con YOLOv8 Orientado a Objetos Pequeños

Agustín Ross

CEIA, Facultad de Ingeniería
Universidad de Buenos Aires
Buenos Aires, Argentina

Ayelén Calo

CEIA, Facultad de Ingeniería
Universidad de Buenos Aires
Buenos Aires, Argentina

María Teresa Mallaupoma León

CEIA, Facultad de Ingeniería
Universidad de Buenos Aires
Buenos Aires, Argentina

Andrés Pablo García

CEIA, Facultad de Ingeniería
Universidad de Buenos Aires
Buenos Aires, Argentina

Resumen—La detección automática de los actores de un partido de fútbol es la primera etapa de cualquier sistema de análisis basado en la transmisión televisiva; sin embargo, la pelota, que ocupa unos 5 px en una entrada de 640×640 , sigue siendo el objeto más difícil de encontrar. En este trabajo se realiza el fine-tuning de YOLOv8s sobre un dataset público curado con cuatro clases (pelota, arquero, jugador y árbitro) y se comparan tres estrategias en idénticas condiciones de entrenamiento: un baseline a 640 px, una entrada de alta resolución a 1280 px y un modelo a 640 px con una cabeza de detección adicional de stride 4 (P2). En test, el modelo a 1280 px eleva el AP50 de la pelota de 0,51 a 0,70 y su recall de 0,48 a 0,73, y obtiene el mejor mAP@50 global (0,747), mientras que la cabeza P2 logra una mejora marginal (AP50 de 0,58) con 1,8 veces la latencia del baseline. Con inferencia en media precisión (FP16), el modelo elegido procesa 43 FPS en una GPU NVIDIA T4 con batch 1 y una pérdida de precisión despreciable, lo que lo hace apto para tiempo real a 25 FPS. Los umbrales de confianza y de NMS se calibran en validación maximizando el F1 macro, y el sistema se demuestra sobre un clip de la Bundesliga. La confusión entre arquero y jugador de campo persiste en todos los modelos y se identifica como una limitación semántica y no espacial.

Palabras clave—detección de objetos, YOLOv8, objetos pequeños, analítica deportiva, fútbol, inferencia en tiempo real

I. INTRODUCCIÓN

Los datos extraídos automáticamente del video de un partido, como las posiciones de los jugadores, la trayectoria de la pelota y los eventos de juego, se han convertido en un insumo central para el análisis táctico, el scouting, la transmisión y el arbitraje en el fútbol profesional. La anotación manual de esta información es lenta y costosa, por lo que los sistemas de visión por computadora que localizan a cada actor relevante en cada cuadro son la base sobre la que se construyen el seguimiento (tracking), la detección de eventos y la analítica de alto nivel.

Un caso real destacado es la tecnología de fuera de juego semiautomático (SAOT) que FIFA utilizó en la Copa Mundial 2022 en Qatar, luego de pruebas en la Copa Árabe de la FIFA 2021 y en el Mundial de Clubes de la FIFA 2021 [1]. El sistema emplea 12 cámaras de seguimiento dedicadas, montadas bajo el techo del estadio, que siguen la pelota y hasta 29 puntos del cuerpo de cada jugador 50 veces por segundo, combinadas con un sensor inercial dentro de la pelota que envía datos 500 veces por segundo. SAOT muestra que la detección basada en visión ya es lo suficientemente madura para apoyar decisiones en el máximo nivel del deporte, pero depende de una infraestructura multicámara dedicada. Los enfoques que trabajan directamente sobre una única señal de transmisión son mucho más económicos y accesibles para clubes y ligas sin esas instalaciones, y benchmarks públicos como SoccerNet-Tracking [2] apuntan precisamente a ese escenario.

El video de transmisión presenta desafíos: en las tomas abiertas la pelota ocupa unos pocos píxeles y suele aparecer borrosa u ocluida, las clases están fuertemente desbalanceadas y los arqueros se distinguen de los jugadores de campo principalmente por el color de la camiseta y la posición. El objetivo de este proyecto es desarrollar y evaluar un detector de los cuatro actores principales (pelota, arquero, jugador y árbitro) en cuadros de transmisión, determinar si una mayor resolución de entrada o una cabeza de detección de alta resolución recupera mejor la pelota bajo un compromiso realista entre precisión y latencia, y aplicar el modelo elegido a un video de partido.

II. METODOLOGÍA, DISEÑO Y DESARROLLO

El sistema se desarrolló en seis etapas, resumidas en la Fig. 1: obtención del dataset, curación y análisis exploratorio, fine-tuning de tres variantes de modelo, evaluación, calibración de los umbrales de posprocesamiento y aplicación a un video de partido. El pipeline completo se implementa en una única notebook de Python basada en la librería Ultralytics y se ejecuta de punta a punta en Google Colab.

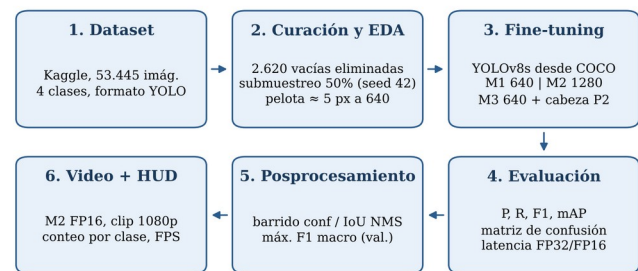


Fig. 1. Etapas del sistema propuesto.

A. Dataset y Curación

Se utiliza el dataset público de fútbol de Kaggle [3], distribuido en formato YOLO (una caja normalizada por línea: clase, x, y, ancho, alto), con 53.445 imágenes divididas en 46.511 de entrenamiento, 4.757 de validación y 2.177 de test. Dado que un cuadro de un partido profesional siempre contiene al menos un actor, las 2.620 imágenes con archivos de etiquetas ausentes o vacíos se eliminaron como errores de anotación. Para poder comparar tres arquitecturas dentro del presupuesto de cómputo de Google Colab, se tomó un submuestreo determinístico del 50% de cada split (lista ordenada y mezcla con semilla 42), con lo que quedaron 25.412 imágenes (22.174 / 2.220 / 1.018). Los splits se guardan como listas de rutas, por lo que los datos originales nunca se modifican. Todas las semillas aleatorias están fijadas y cuDNN opera en modo determinístico.

B. Análisis Exploratorio

El análisis de 37.118 cajas de 2.000 imágenes de validación confirma un desbalance severo: los jugadores representan el 85,8% de las instancias, los árbitros el 7,5%, la

pelota el 4,3% y los arqueros apenas el 2,5%. A 640×640 , la caja mediana de la pelota mide $4,0 \times 7,5$ px (lado medio de 5,8 px), frente a 9×35 px de los jugadores. La grilla de detección más fina de YOLOv8 estándar tiene un stride de 8 px, por lo que la pelota es más chica que una sola celda de la grilla (Fig. 2); esta observación guía el diseño de los modelos.

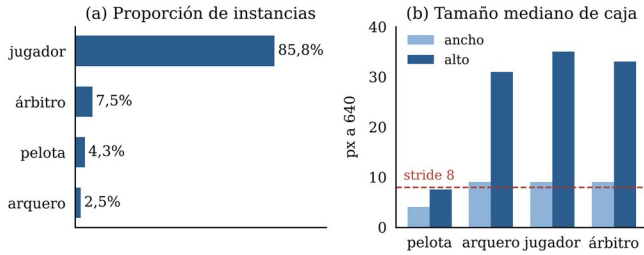


Fig. 2. Análisis exploratorio del subconjunto de validación: (a) desbalance de clases; (b) ancho y alto medianos de las cajas a 640×640 , comparados con el stride más fino de YOLOv8.

C. Variantes de Modelo

Todos los modelos parten de YOLOv8s [4], un detector de una sola etapa y sin anchors de la familia YOLO [5], preentrenado en MS COCO [6] y ajustado de punta a punta (sin capas congeladas) con una nueva cabeza de cuatro clases. Su neck combina una pirámide de features [7] y una red de agregación de caminos [8], con cabezas de detección en los niveles P3, P4 y P5 (strides 8, 16 y 32). La literatura sobre objetos pequeños propone varias soluciones, como sobremuestrear y aumentar las instancias pequeñas [9], dividir las imágenes en mosaicos superpuestos [10], agrandar la entrada o agregar niveles más finos a la pirámide. Se comparan estas dos últimas, que no requieren cambios en el pipeline de datos:

- **M1 (baseline):** YOLOv8s con entradas de 640 px.
- **M2 (alta resolución):** la misma red con entradas de 1280 px. Las imágenes se reescalan al vuelo con letterbox y las etiquetas no cambian porque están normalizadas; la pelota pasa a medir unos 10 px y todas las capas operan con el doble de resolución espacial, con un costo de cómputo cercano a cuatro veces mayor.
- **M3 (cabeza P2):** la configuración yolov8s-p2 a 640 px, que extiende el camino descendente hasta stride 4 y agrega una cuarta cabeza sobre una grilla de 160×160 . Los pesos del backbone se transfieren desde COCO; las capas nuevas parten de una inicialización aleatoria.

D. Configuración del Entrenamiento

Los tres modelos comparten hiperparámetros idénticos: 15 épocas, batch de 16, early stopping con paciencia de 8 épocas y la configuración por defecto del optimizador de Ultralytics. El data augmentation se adaptó al fútbol: flip horizontal con probabilidad 0,5 (la cancha es simétrica), sin flip vertical (las cámaras de transmisión mantienen una orientación fija), variaciones HSV ($h = 0,015$; $s = 0,7$; $v = 0,4$) y mosaic, desactivado en las últimas tres épocas para estabilizar la regresión de cajas. Se guardaron checkpoints en Google Drive cada cinco épocas para poder reanudar sesiones de Colab interrumpidas. Solo la resolución de entrada y la arquitectura difieren entre modelos, por lo que las diferencias pueden atribuirse a esos factores.

E. Protocolo de Evaluación

Los modelos se comparan en validación y test con precisión (P), recall (R), F1, mAP@50, mAP@75 y mAP@50–95, AP50 por clase y recall de la pelota, además de

matrices de confusión normalizadas por clase real. Como las herramientas de validación miden el tiempo en lotes de imágenes, la latencia se midió aparte en condiciones de video: 200 cuadros del clip procesados de a uno (batch 1) luego de 20 cuadros de calentamiento, con sincronización de CUDA alrededor de cada llamada, en FP32 y FP16, sobre una NVIDIA T4. A 25 FPS llega un cuadro cada 40 ms, lo que define el criterio de tiempo real.

F. Calibración del Posprocesamiento

Para el modelo elegido se ajustaron el umbral de confianza y el umbral de IoU de la supresión de no máximos (NMS). Para cada IoU de NMS en $\{0,5; 0,6; 0,7; 0,8\}$, el modelo se ejecuta una sola vez con confianza 0,01; las detecciones se ordenan por confianza y se emparejan de forma greedy con cajas reales no usadas de la misma clase con $\text{IoU} \geq 0,5$. Luego se evalúan confianzas de 0,05 a 0,90 filtrando esa tabla, sin volver a ejecutar la red. Se elige en validación la configuración con mayor F1 macro (promedio de los F1 de las cuatro clases, que le da a la pelota el mismo peso que al jugador) y se confirma una única vez en test.

G. Aplicación en Video

La configuración elegida procesa cuadro a cuadro, con OpenCV, un clip de 30 s del DFL Bundesliga Data Shootout [11] (1080p, 25 FPS). Se activa el NMS agnóstico a la clase para que una persona no reciba dos etiquetas superpuestas (por ejemplo, jugador y arquero). Cada cuadro se anota con cajas coloreadas por clase y un HUD que muestra el conteo por clase, la presencia de la pelota y los FPS suavizados exponencialmente, y se escribe en un archivo MP4.

III. RESULTADOS EXPERIMENTALES Y ANÁLISIS

A. Precisión de Detección

Las curvas de validación muestran que los tres modelos alcanzan una meseta en las últimas épocas, con M2 por encima de los demás durante todo el entrenamiento (mAP@50 final en validación de 0,786 frente a 0,733 de M1 y 0,737 de M3). La Tabla I presenta los resultados en test. M2 obtiene el mejor mAP@50, mAP@50–95 y F1 globales, y la mayor mejora se da en la pelota: el AP50 sube de 0,51 a 0,70 y el recall de 0,48 a 0,73. Es decir, M1 pierde más de la mitad de las pelotas y M2 aproximadamente una de cada cuatro. M3 mejora el AP50 de la pelota en apenas 0,07.

TABLA I. RESULTADOS EN TEST DE LOS TRES MODELOS

Modelo	P	R	F1	mAP @50	mAP @50–95	AP50 pelota	R pelota	AP50 arb.	AP50 ár.
M1 640	0,807	0,780	0,793	0,709	0,439	0,505	0,485	0,714	0,641
M2 1280	0,775	0,847	0,810	0,747	0,466	0,698	0,732	0,692	0,613
M3 P2	0,803	0,776	0,789	0,713	0,447	0,577	0,526	0,669	0,632

arb.: arquero; ár.: árbitro. Umbrales por defecto de Ultralytics; FP32.

La mayor resolución tiene un costo: en test, M2 pierde precisión (0,775 frente a 0,807) y AP50 en el árbitro (0,613 frente a 0,641), lo que indica más falsos positivos; esto se aborda con la calibración de umbrales. El mAP@50–95 de la pelota es bajo en todos los modelos (0,26–0,30) por una limitación de la métrica en objetos diminutos: en una caja de 5×5 px, un desplazamiento de 1 px en ambos ejes reduce el IoU a $16/34 \approx 0,47$, por debajo incluso del umbral más permisivo.

B. Análisis de Errores

Clase predicha	(a) M1 (640 px)					(b) M2 (1280 px)				
	pelota	arq.	jug.	arb.	fondo	pelota	arq.	jug.	arb.	fondo
	0,61	0,01	0,00	0,00	0,15	0,83	0,00	0,00	0,00	0,20
	0,00	0,44	0,02	0,01	0,08	0,00	0,57	0,01	0,01	0,06
	0,02	0,54	0,96	0,11	0,66	0,02	0,42	0,97	0,09	0,62
	0,00	0,00	0,01	0,74	0,11	0,00	0,00	0,01	0,75	0,12
Clase real	0,37	0,01	0,01	0,14	0,00	0,15	0,01	0,00	0,14	0,00
	pelota	arq.	jug.	arb.	fondo	pelota	arq.	jug.	arb.	fondo

Fig. 3. Matrices de confusión en test normalizadas por clase real (columnas). arq.: arquero; jug.: jugador; arb.: árbitro; fondo: objetos no detectados en la última fila y falsos positivos en la última columna.

La Fig. 3 confirma los dos modos de error principales. Las pelotas no detectadas (pelota → fondo) bajan del 37% con M1 al 15% con M2. Los arqueros son la principal confusión entre clases: M1 clasifica como jugador al 54% y M2 todavía al 42%, y el AP50 del arquero no mejora con la resolución (0,67–0,71 en los tres modelos). Un arquero ya ocupa varias celdas de la grilla, por lo que la dificultad es semántica, ya que se distingue del jugador de campo principalmente por la camiseta y el contexto, y se agrava por ser la clase más escasa. Los árbitros tienen un rendimiento razonable (14% no detectados y 9–11% clasificados como jugador). La mayoría de los falsos positivos sobre el fondo (62–66%) se etiquetan como jugador, lo que es consistente con suplentes, cuerpo técnico y público cerca de la línea de banda.

C. Latencia

TABLA II. LATENCIA EXTREMO A EXTREMO POR CUADRO (GPU T4, BATCH 1)

Modelo	FP32 (ms)	FP16 (ms)	FP16 p95 (ms)	FPS FP32	FPS FP16	Tiempo real 25 FPS
M1 640	16,8	8,9	10,7	59,6	112,5	Sí
M2 1280	60,7	23,4	25,6	16,5	42,8	Solo FP16
M3 P2	29,8	11,3	16,3	33,6	88,2	Sí

La Tabla II muestra que M2 no alcanza tiempo real en FP32 (60,7 ms), pero FP16 reduce su latencia a 23,4 ms (42,8 FPS; percentil 95 de 25,6 ms), con margen para la decodificación y el dibujado. FP16 acelera los tres modelos entre 1,9 y 2,6 veces gracias a los tensor cores de la T4. M3 es unas 1,8 veces más lento que M1 en FP32 por la cabeza adicional de alta resolución. Al reevaluar M2 en FP16 sobre test, el mAP@50 pasa de 0,7469 a 0,7458 y el AP50 de la pelota de 0,698 a 0,693; el mayor efecto es sobre el recall de la pelota (de 0,732 a 0,692), que aun así supera ampliamente a M1 y M3. Por lo tanto, se eligió M2 en FP16.

D. Calibración de Umbrales

En validación, la mejor configuración es confianza 0,30 con IoU de NMS 0,5 (F1 macro de 0,828). El F1 macro se mantiene entre 0,823 y 0,828 para confianzas en [0,30; 0,40] e IoU en [0,5; 0,7], y cae a 0,810 con IoU 0,8, por lo que el sistema es robusto a la elección exacta. En test, los umbrales elegidos dan un F1 por clase de 0,746 para la pelota (P 0,824; R 0,682), 0,806 para arqueros, 0,963 para jugadores y 0,748 para árbitros. Esta evaluación usa NMS por clase, que puede conservar una caja de jugador y otra de arquero sobre la misma persona y así favorecer el recall del arquero (0,909); en la aplicación en video se usa NMS agnóstico a la clase para evitar esos duplicados.

E. Aplicación en Video



Fig. 4. Cuadro anotado del clip de la Bundesliga producido por M2 (FP16): jugadores (verde), arquero (cian), árbitros (rojo) y pelota (amarillo), con el HUD en la parte superior.

El procesamiento del clip 1080p toma en promedio 30,1 ms por cuadro, incluyendo inferencia y dibujado, lo que equivale a 33 FPS y supera los 25 FPS del video original. La Fig. 4 muestra un cuadro representativo: se detectan todos los actores en la cancha y la pelota en juego abierto, en concordancia con los resultados cuantitativos.

F. Limitaciones

La comparación se basa en una única corrida de entrenamiento por modelo, sobre el 50% de los datos y 15 épocas, por lo que diferencias pequeñas, como las observadas entre M1 y M3, deben interpretarse con cautela en ausencia de estimaciones de varianza. El modelo de alta resolución lleva cada imagen a 1280 px; cuando la imagen original es más chica, la red ve una imagen agrandada, de modo que su ventaja proviene de la grilla efectiva más fina y no de detalle adicional. La latencia se midió en una T4 compartida de Colab, donde los tiempos varían algunos milisegundos entre sesiones, aunque las conclusiones se mantuvieron en todas las corridas. Por último, el sistema detecta objetos de forma independiente en cada cuadro y todavía no aprovecha la información temporal, que podría recuperar pelotas perdidas en cuadros aislados.

IV. CONCLUSIONES

1) *La resolución de entrada es la principal palanca para detectar la pelota.* Duplicar la entrada a 1280 px elevó el AP50 de la pelota de 0,51 a 0,70 y redujo las pelotas perdidas del 37% al 15%, lo que confirma que el problema es el tamaño del objeto respecto de la grilla de detección.

2) *Una cabeza de stride 4 no es un sustituto eficiente.* La variante P2 mejoró el AP50 de la pelota en solo 0,07 con 1,8 veces la latencia del baseline. Una explicación plausible es que las features de stride 4 provienen de capas tempranas con poco contenido semántico, mientras que una entrada más grande aumenta la resolución de toda la red.

3) *La media precisión hace utilizable en tiempo real al modelo más preciso.* FP16 aceleró M2 2,6 veces, hasta 43 FPS en una T4, con un cambio de 0,001 en el mAP@50; la caída de 0,04 en el recall de la pelota debe monitorearse.

4) *La confusión del arquero es semántica y está impulsada por el desbalance.* Incluso a 1280 px, el 42% de los arqueros se etiqueta como jugador. Entre las soluciones posibles están sumar ejemplos de arqueros, el muestreo o la ponderación de la pérdida por clase, y pistas de contexto como el color de la camiseta o la posición cerca del arco.

5) *Una curación cuidadosa y la reproducibilidad permiten comparaciones justas.* Eliminar 2.620 imágenes sin etiquetas,

submuestrear de forma determinística y fijar todas las semillas produjo splits y métricas idénticos entre sesiones, y redujo a la mitad el costo de entrenamiento.

6) *Las métricas y los umbrales deben contemplar los objetos diminutos.* El mAP@50–95 de la pelota está dominado por errores de localización de 1 px, por lo que el AP50 y el recall son más informativos para esa clase; la calibración por F1 macro mostró un óptimo plano, y se adoptaron confianza 0,30 e IoU de NMS 0,5.

Como trabajo futuro, el detector se combinará con un algoritmo de seguimiento multiobjeto como ByteTrack [12] para mantener la identidad de los jugadores e interpolar la pelota cuando no se detecta, y se explorarán la asignación de equipos, el procesamiento por mosaicos y backbones más grandes entrenados con el dataset completo.

REFERENCIAS

- [1] FIFA, “Semi-automated offside technology,” FIFA Inside, 2022. [En línea]. Disponible en: <https://inside.fifa.com/technical/football-technology/football-technologies-and-innovations-at-the-fifa-world-cup-2022/semi-automated-offside-technology>
- [2] A. Cioppa et al., “SoccerNet-Tracking: Multiple object tracking dataset and benchmark in soccer videos,” en *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2022.
- [3] “Football dataset,” Kaggle (usuario oussamamoussa). [En línea]. Disponible en: <https://www.kaggle.com/datasets/oussamamoussa/foot-ball-dataset>
- [4] G. Jocher, A. Chaurasia y J. Qiu, “Ultralytics YOLOv8,” repositorio de GitHub, 2023. [En línea]. Disponible en: <https://github.com/ultralytics/ultralytics>
- [5] J. Redmon, S. Divvala, R. Girshick y A. Farhadi, “You only look once: Unified, real-time object detection,” en *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [6] T.-Y. Lin et al., “Microsoft COCO: Common objects in context,” en *Proc. European Conf. Computer Vision (ECCV)*, 2014, pp. 740–755.
- [7] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan y S. Belongie, “Feature pyramid networks for object detection,” en *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2117–2125.
- [8] S. Liu, L. Qi, H. Qin, J. Shi y J. Jia, “Path aggregation network for instance segmentation,” en *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 8759–8768.
- [9] M. Kisanal, Z. Wojna, J. Murawski, J. Naruniec y K. Cho, “Augmentation for small object detection,” 2019, arXiv:1902.07296.
- [10] F. C. Akyon, S. O. Altinuc y A. Temizel, “Slicing aided hyper inference and fine-tuning for small object detection,” en *Proc. IEEE Int. Conf. Image Processing (ICIP)*, 2022, pp. 966–970.
- [11] DFL Deutsche Fußball Liga, “DFL – Bundesliga Data Shootout,” competencia de Kaggle, 2022. [En línea]. Disponible en: <https://www.kaggle.com/competitions/dfl-bundesliga-data-shootout>
- [12] Y. Zhang et al., “ByteTrack: Multi-object tracking by associating every detection box,” en *Proc. European Conf. Computer Vision (ECCV)*, 2022.